Advanced Python

50 Questions & Answers • Code, Outputs, and Explanations

Practice guide based on the uploaded “Python Practice Set — Second Batch” PDF. The source contains 49 numbered questions; Question 50 is an additional practice question included to complete the set.

Topics

Functions • Recursion • Scope and Closures • Decorators • Generators • Iteration • Functional Tools • Exceptions • Context Managers • Mixed Challenges • Debugging

Q1. Predict the output: default and positional arguments

```
def f(x, y=2):
    return x ** y

print(f(3))
print(f(3, 3))
```

Output:

```
9
27
```

Explanation: Default y is 2; the second call supplies y=3.

Q2. Write a function using *args that returns the sum

```
def total(*args):
    return sum(args)

print(total(10, 20, 30))
```

Output:

```
60
```

Explanation: *args collects positional arguments into a tuple.

Q3. Write a function using **kwargs

```
def student(**kwargs):
    for key, value in kwargs.items():
        print(key, ":", value)

student(name="Tanmay", age=20, course="Python")
```

Output:

```
name : Tanmay
age : 20
course : Python
```

Explanation: **kwargs collects keyword arguments into a dictionary.

Q4. Explain positional, keyword, default, and keyword-only arguments

```
def student(name, age=20, *, city):
    print(name, age, city)

student("Tanmay", city="Pune")
```

Output:

```
Tanmay 20 Pune
```

Explanation: Positional arguments are assigned by position; keyword arguments by name; default arguments use a fallback value; parameters after * are keyword-only.

Q5. Recursive factorial

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5))
```

Output:

```
120
```

Explanation: 5 × 4 × 3 × 2 × 1 = 120.

Q6. Recursive Fibonacci number

```
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

print(fibonacci(6))
```

Output:

```
8
```

Explanation: The sequence begins 0, 1, 1, 2, 3, 5, 8.

Q7. Closure example

```
def outer(x):
    def inner(y):
        return x + y
    return inner

f = outer(10)
print(f(5))
```

Output:

```
15
```

Explanation: The inner function remembers x=10 from the outer function.

Q8. What is a closure?

```
def outer():
    x = 10
    def inner():
        return x
    return inner

f = outer()
print(f())
```

Output:

```
10
```

Explanation: A closure is an inner function that retains access to variables from its enclosing function after that function has returned.

Q9. Predict the output using nonlocal

```
x = 10

def outer():
    x = 20
    def inner():
        nonlocal x
        x += 5
    inner()
    return x

print(outer())
```

Output:

```
25
```

Explanation: nonlocal modifies the variable in the enclosing function.

Q10. Explain LEGB

LEGB is Python's variable lookup order: Local (current function), Enclosing (outer function), Global (module), Built-in (Python built-ins such as print).

Q11. Decorator that prints Starting and Finished

```
def my_decorator(func):
    def wrapper():
        print("Starting")
        func()
        print("Finished")
    return wrapper

@my_decorator
def hello():
    print("Hello Python")

hello()
```

Output:

```
Starting
Hello Python
Finished
```

Explanation: The wrapper runs extra statements before and after the decorated function.

Q12. What is a decorator?

A decorator modifies or extends a function's behavior without changing its original code. A wrapper commonly runs extra code before or after the original function.

Q13. Predict the decorator output

```
def deco(fn):
    def wrap():
        print("A")
        fn()
        print("B")
    return wrap

@deco
def hello():
    print("Hi")

hello()
```

Output:

```
A
Hi
B
```

Explanation: The decorator's wrapper prints A, calls hello, then prints B.

Q14. Decorator that measures execution time

```
import time

def timer(func):
    def wrapper():
        start = time.perf_counter()
        func()
        end = time.perf_counter()
        print("Time:", end - start, "seconds")
    return wrapper

@timer
def work():
    print("Working")

work()
```

Output:

```
Working
Time: [execution time] seconds
```

Explanation: The exact elapsed time varies; perf_counter measures elapsed time.

Q15. Decorator supporting *args and **kwargs

```
def decorator(func):
    def wrapper(*args, **kwargs):
        print("Function started")
        return func(*args, **kwargs)
    return wrapper

@decorator
def add(a, b):
    return a + b

print(add(10, 20))
```

Output:

```
Function started
30
```

Explanation: The wrapper forwards any positional and keyword arguments.

Q16. Why use functools.wraps?

functools.wraps preserves the original function's metadata, such as its name and docstring, when decorating it.

Q17. Decorator allowing only admins

```
def admin_required(func):
    def wrapper(user_is_admin):
        if user_is_admin:
            return func(user_is_admin)
        return "Access Denied"
    return wrapper

@admin_required
def dashboard(user_is_admin):
    return "Welcome Admin"

print(dashboard(True))
print(dashboard(False))
```

Output:

```
Welcome Admin
Access Denied
```

Explanation: The decorated function runs only when user_is_admin is True.

Q18. Decorator that retries up to three times

```
def retry(func):
    def wrapper():
        for attempt in range(3):
            try:
                return func()
            except Exception:
                print("Attempt", attempt + 1, "failed")
        print("All attempts failed")
    return wrapper

@retry
def task():
    print("Task completed")

task()
```

Output:

```
Task completed
```

Explanation: The function is attempted up to three times when it raises an exception; this sample succeeds on its first attempt.

Q19. @decorator syntax vs manual call

These are equivalent: @decorator above a function, and defining the function then assigning function_name = decorator(function_name).

Q20. Generator return value and StopIteration

```
def gen():
    yield 1
    yield 2
    return 3

g = gen()
print(next(g))
print(next(g))
try:
    print(next(g))
except StopIteration as e:
    print("Done", e.value)
```

Output:

```
1
2
Done 3
```

Explanation: The return value of a finished generator is carried by StopIteration.value.

Q21. Difference between yield and return

yield produces a value and pauses a generator while preserving its state. return ends a function and returns its result.

Q22. Generator yielding even numbers

```
def even_numbers(limit):
    for i in range(limit + 1):
        if i % 2 == 0:
            yield i

print(list(even_numbers(10)))
```

Output:

```
[0, 2, 4, 6, 8, 10]
```

Explanation: The generator yields even numbers through the inclusive limit.

Q23. Exhausted generator and next(g, default)

```
def gen():
    yield 10

g = gen()
print(next(g, "Finished"))
print(next(g, "Finished"))
```

Output:

```
10
Finished
```

Explanation: An exhausted generator raises StopIteration; next(g, default) returns the default instead.

Q24. Why generators are lazy

Generators produce values only when requested. This can save memory and allow one-at-a-time processing instead of building a full list.

Q25. Generator expression for squares 1 through 10

```
squares = (x ** 2 for x in range(1, 11))
print(list(squares))
```

Output:

```
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

Explanation: A generator expression computes each square as it is consumed.

Q26. Generator cleanup after close()

```
def gen():
    try:
        yield "A"
        yield "B"
    finally:
        print("Cleanup")

g = gen()
print(next(g))
g.close()
```

Output:

```
A
Cleanup
```

Explanation: Closing the generator triggers its finally block.

Q27. When does a generator finally block execute?

It executes when the generator finishes, is explicitly closed, or is finalized during garbage collection. Garbage-collection timing is not guaranteed.

Q28. Safely close after consuming first value

```
def gen():
    try:
        yield 10
        yield 20
    finally:
        print("Generator closed")

g = gen()
try:
    print(next(g))
finally:
    g.close()
```

Output:

```
10
Generator closed
```

Explanation: The finally block ensures close() runs even if consuming the value raises an exception.

Q29. Iterable vs iterator vs generator

An iterable can provide an iterator; an iterator yields values through next(); a generator is an iterator created with yield or a generator expression.

Q30. Predict iterator output

```
nums = [1, 2, 3]
it = iter(nums)
print(next(it))
print(next(it))
```

Output:

```
1
2
```

Explanation: Each next() advances the iterator.

Q31. Custom iterator from 1 through n

```
class Numbers:
    def __init__(self, n):
        self.n = n
        self.current = 1

    def __iter__(self):
        return self

    def __next__(self):
        if self.current <= self.n:
            value = self.current
            self.current += 1
            return value
        raise StopIteration

for num in Numbers(5):
    print(num)
```

Output:

```
1
2
3
4
5
```

Explanation: __iter__ returns the iterator and __next__ supplies values until StopIteration.

Q32. Explain map(), filter(), and reduce()

```
from functools import reduce

nums = [1, 2, 3, 4, 5]
print(list(map(lambda x: x ** 2, nums)))
print(list(filter(lambda x: x % 2 == 0, nums)))
print(reduce(lambda x, y: x + y, nums))
```

Output:

```
[1, 4, 9, 16, 25]
[2, 4]
15
```

Explanation: map transforms, filter selects, and reduce combines items into one result.

Q33. Sort tuples by word length

```
fruits = [("apple", 5), ("fig", 2), ("banana", 6)]
result = sorted(fruits, key=lambda x: len(x[0]))
print(result)
```

Output:

```
[('fig', 2), ('apple', 5), ('banana', 6)]
```

Explanation: The key function measures the length of the first tuple element.

Q34. Late-binding lambda output

```
funcs = [lambda: i for i in range(3)]
print([f() for f in funcs])
```

Output:

```
[2, 2, 2]
```

Explanation: Each lambda looks up the same i after the loop has finished.

Q35. Fix late-binding lambda

```
funcs = [lambda i=i: i for i in range(3)]
print([f() for f in funcs])
```

Output:

```
[0, 1, 2]
```

Explanation: The default argument captures each iteration's value.

Q36. Shallow copy vs deep copy

```
import copy

original = [[1, 2], [3, 4]]
shallow = copy.copy(original)
deep = copy.deepcopy(original)

original[0][0] = 100
print(shallow)
print(deep)
```

Output:

```
[[100, 2], [3, 4]]
[[1, 2], [3, 4]]
```

Explanation: A shallow copy shares nested objects; a deep copy recursively copies them.

Q37. try-except-else-finally division

```
try:
    a = 10
    b = 0
    result = a / b
except ZeroDivisionError:
    print("Cannot divide by zero")
else:
    print("Result:", result)
finally:
    print("Program completed")
```

Output:

```
Cannot divide by zero
Program completed
```

Explanation: else runs only without an exception; finally runs regardless.

Q38. Raise exception vs return error value

Raising an exception interrupts normal flow and requires handling. Returning an error value is a normal return, so the caller must check it.

Q39. Class-based context manager

```
class MyContext:
    def __enter__(self):
        print("Entering")
        return self

    def __exit__(self, exc_type, exc_value, traceback):
        print("Exiting")
        return False

with MyContext():
    print("Inside")
```

Output:

```
Entering
Inside
Exiting
```

Explanation: __enter__ starts the context and __exit__ performs cleanup.

Q40. What does with do?

```
with open("sample.txt", "w") as file:
    file.write("Hello Python")
```

Explanation: The with statement manages resources and automatically closes the file when the block ends.

Q41. Return inside finally

```
def f():
    try:
        return "try"
    finally:
        return "finally"

print(f())
```

Output:

```
finally
```

Explanation: A return in finally overrides the return from try.

Q42. Short-circuit evaluation

```
x = 0
print(x != 0 and 10 / x > 1)
print(x == 0 or 10 / x > 1)
```

Output:

```
False
True
```

Explanation: and skips the second expression when its first operand is False; or skips it when the first operand is True.

Q43. Password validation using Boolean conditions

```
def validate_password(password):
    return (
        len(password) >= 8
        and any(c.isupper() for c in password)
        and any(c.islower() for c in password)
        and any(c.isdigit() for c in password)
    )

print(validate_password("Python123"))
print(validate_password("python"))
```

Output:

```
True
False
```

Explanation: Checks length, uppercase, lowercase, and digit.

Q44. Menu-driven program using functions, decorators, and exceptions

```
def log(func):
    def wrapper():
        print("Executing operation")
        return func()
    return wrapper

@log
def add():
    try:
        a = int(input("Enter first number: "))
        b = int(input("Enter second number: "))
        print("Sum:", a + b)
    except ValueError:
        print("Enter valid numbers")

def menu():
    while True:
        print("\\n1. Add")
        print("2. Exit")
        choice = input("Enter choice: ")
        if choice == "1":
            add()
        elif choice == "2":
            print("Goodbye")
            break
        else:
            print("Invalid choice")

menu()
```

Explanation: Uses functions, a decorator, a loop, conditions, and exception handling.

Q45. Generator pipeline: numbers, evens, squares

```
def numbers():
    for n in range(1, 11):
        yield n

def even_numbers():
    for n in numbers():
        if n % 2 == 0:
            yield n

def squares():
    for n in even_numbers():
        yield n ** 2

print(list(squares()))
```

Output:

```
[4, 16, 36, 64, 100]
```

Explanation: The pipeline yields numbers, filters even values, then squares them.

Q46. Decorator that caches results

```
def cache(func):
    saved = {}
    def wrapper(n):
        if n not in saved:
            saved[n] = func(n)
        return saved[n]
    return wrapper

@cache
def square(n):
    print("Calculating...")
    return n * n

print(square(5))
print(square(5))
```

Output:

```
Calculating...
25
25
```

Explanation: The second call reuses the cached result. A simple cache uses memory and here supports only one hashable argument.

Q47. nonlocal vs global

```
x = 10

def outer():
    x = 20
    def inner():
        nonlocal x
        x = 30
    inner()
    print("Enclosing:", x)

outer()
print("Global:", x)
```

Output:

```
Enclosing: 30
Global: 10
```

Explanation: nonlocal modifies an enclosing function variable; global modifies a module-level variable.

Q48. Command-line quiz application

```
def quiz():
    questions = {
        "What is the output of 2 ** 3?": "8",
        "Which keyword creates a function?": "def",
        "Which keyword creates a generator?": "yield"
    }
    score = 0
    for question, answer in questions.items():
        print(question)
        user_answer = input("Answer: ")
        if user_answer.lower() == answer.lower():
            print("Correct!")
            score += 1
        else:
            print("Wrong! Answer:", answer)
    print("Final Score:", score, "/", len(questions))

quiz()
```

Explanation: The dictionary stores Q&A; the loop asks questions, conditions check answers, and score tracks correct responses.

Q49. Debug late-binding function list

```
def make_funcs():
    funcs = []
    for i in range(3):
        def f():
            return i
        funcs.append(f)
    return funcs

print([f() for f in make_funcs()])
```

Output:

```
[2, 2, 2]
```

Explanation: All functions reference the same i, which is 2 after the loop. Fix with def f(i=i): return i.

Q50. Additional practice: Fibonacci generator up to a limit

```
def fibonacci(limit):
    a, b = 0, 1
    while a <= limit:
        yield a
        a, b = b, a + b

print(list(fibonacci(50)))
```

Output:

```
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

Explanation: This is an extra question added because the uploaded PDF has 49 numbered questions.

Quick Revision

Topic	Remember
*args	Multiple positional arguments
**kwargs	Multiple keyword arguments
Recursion	Function calls itself
Closure	Inner function remembers outer variables
nonlocal	Modify enclosing variable
Decorator	Modify function behavior
Generator	Produces values using yield
Iterator	Advances with next()
map()	Transform values
filter()	Select values
reduce()	Combine values
finally	Cleanup code runs
Context manager	Manages resources
Short-circuit	Skips unnecessary Boolean evaluation
Cache	Stores prior results